

pgAdmin 4

File Object Tools Edit View Window Help

server (4)

- MegaSena-Docker
- PostgreSQL 14
- PostgreSQL 18
 - Databases (11)
 - BD_AULA
 - agrislab
 - bd_escola
 - biblioteca
 - clima_alerta
 - elevador
 - focosdb
 - petshop_db
 - postgres
 - queimadas_db
 - sistema_bancario
 - Login/Group Roles
 - Tablespaces
 - Postgres Enterprise Ma

biblioteca/postgres@PostgreSQL 18

Query Query History

```
1 CREATE OR REPLACE FUNCTION bloquear_exclusao()
2 RETURNS TRIGGER LANGUAGE plpgsql AS $$
3 BEGIN
4     IF OLD.quantidade > 0 THEN
5         RAISE EXCEPTION ;
6     END IF;
7     RETURN OLD;
8 END;
9 $$;
10
11 CREATE TRIGGER trg_bloquear_exclusao
12 BEFORE DELETE ON livro
13 FOR EACH ROW
14 EXECUTE FUNCTION bloquear_exclusao();
15
16
17 CREATE TABLE IF NOT EXISTS log_exclusao (
18     id SERIAL PRIMARY KEY,
19     titulo VARCHAR(255),
20     data_exclusao TIMESTAMP,
21     mensagem TEXT
22 );
23
24
25 CREATE OR REPLACE FUNCTION log_exclusao_livro()
26 RETURNS TRIGGER LANGUAGE plpgsql AS $$
```

Total rows: Query complete 00:00:00.191

CRLF Ln 45, Col 42

15:08 16/06/2026

pgAdmin 4

File Object Tools Edit View Window Help

server (4)

- MegaSena-Docker
- PostgreSQL 14
- PostgreSQL 18
 - Databases (11)
 - BD_AULA
 - agrislab
 - bd_escola
 - biblioteca
 - clima_alerta
 - elevador
 - focosdb
 - petshop_db
 - postgres
 - queimadas_db
 - sistema_bancario
 - Login/Group Roles
 - Tablespaces
 - Postgres Enterprise Ma

biblioteca/postgres@PostgreSQL 18

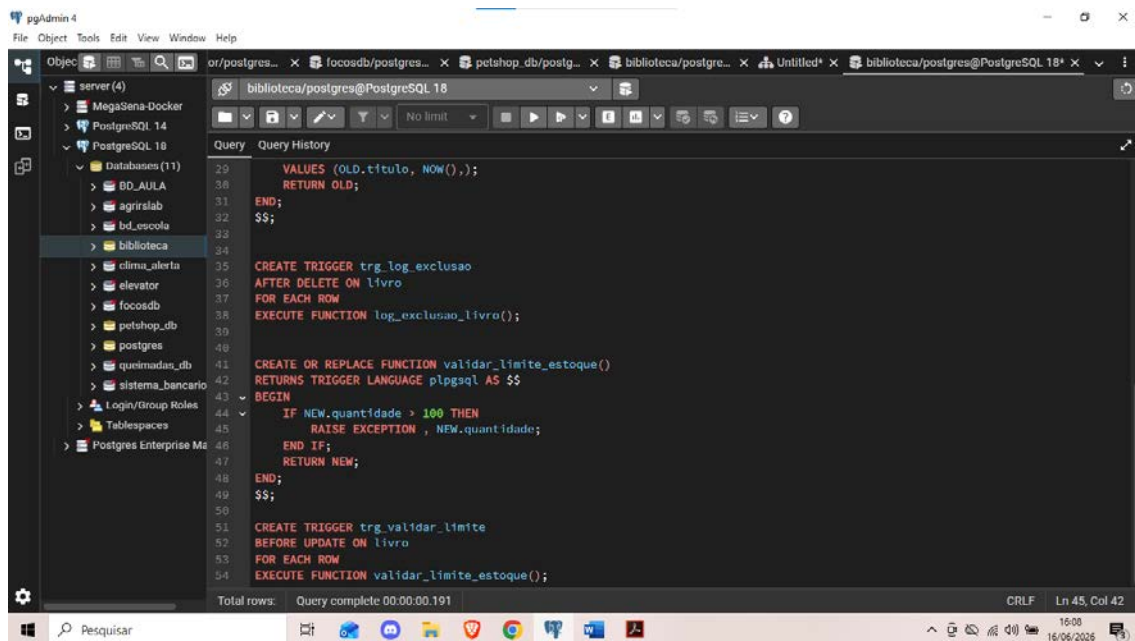
Query Query History

```
24
25 CREATE OR REPLACE FUNCTION log_exclusao_livro()
26 RETURNS TRIGGER LANGUAGE plpgsql AS $$
27 BEGIN
28     INSERT INTO log_exclusao (titulo, data_exclusao, mensagem)
29     VALUES (OLD.titulo, NOW(),);
30     RETURN OLD;
31 END;
32 $$;
33
34
35 CREATE TRIGGER trg_log_exclusao
36 AFTER DELETE ON livro
37 FOR EACH ROW
38 EXECUTE FUNCTION log_exclusao_livro();
39
40
41 CREATE OR REPLACE FUNCTION validar_limite_estoque()
42 RETURNS TRIGGER LANGUAGE plpgsql AS $$
43 BEGIN
44     IF NEW.quantidade > 100 THEN
45         RAISE EXCEPTION , NEW.quantidade;
46     END IF;
47     RETURN NEW;
48 END;
49 $$;
```

Total rows: Query complete 00:00:00.191

CRLF Ln 48, Col 5

15:08 16/06/2026



Exercício 4:

- **a) A diferença entre triggers BEFORE e AFTER:** A trigger BEFORE é acionada *antes* da instrução SQL (como um UPDATE ou DELETE) ser concluída, permitindo barrar a ação ou modificar os dados antes de irem para o banco. A trigger AFTER é acionada *depois* que os dados já foram alterados no banco, servindo para disparar ações em cascata baseadas no que acabou de acontecer.
- **b) Qual delas deve ser usada para validação:** BEFORE. Se a validação falhar, você usa um RAISE EXCEPTION para abortar a operação antes que o banco seja modificado incorretamente.
- **c) Qual delas deve ser usada para auditoria:** AFTER. Você só quer registrar um log de auditoria se a operação principal realmente deu certo.
- **d) Por que a ordem de execução é importante:** Garante a consistência. Se você registrar um log BEFORE e a operação principal falhar logo depois, seu sistema terá um log fantasma de uma exclusão que nunca aconteceu na realidade.

Exercício 5: Reflexão sobre Integridade e Automação

- **a) Riscos de remover do banco e deixar na aplicação:** Perda de consistência. Se houver mais de um sistema acessando o mesmo banco de dados (ex: um app e um site), e apenas um deles tiver a regra implementada, o outro poderá inserir dados inválidos.
- **b) Vantagens de manter regras no banco:** As regras ficam centralizadas e à prova de falhas na camada de aplicação. Nenhuma inserção, nem mesmo um script manual via pgAdmin, consegue burlar uma trigger.
- **c) Como ajudam na integridade e consistência:** Triggers aplicam regras de negócio complexas de forma automática e atômica. Elas garantem que, se uma condição não for atendida, o dado não entra ou não é alterado, mantendo a base de dados sempre confiável.

- **d) Exemplo corporativo real:** Em um e-commerce, ao atualizar o status de um pedido para "Cancelado" (tabela pedido), uma trigger AFTER UPDATE devolve imediatamente a quantidade comprada para a tabela estoque_produtos, garantindo que os produtos voltem à prateleira virtual sem depender de uma requisição extra da API.